PRACTICAL-2

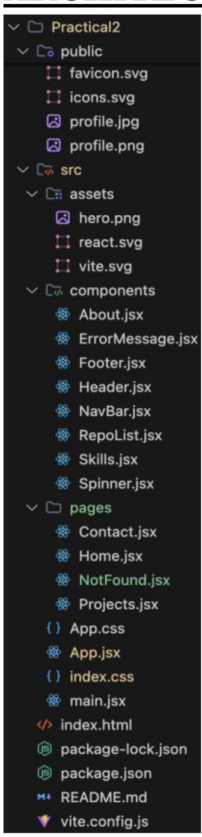
PROBLEM DEFINITION:

- Extend the portfolio application from Practical 1 by adding React Router.
- Implement a navigation bar with at least 3 routes: Home, Projects, and Contact.
- Each route must render a distinct component without a full page reload.
- Add at least 2 useState variables used meaningfully one for toggling UI visibility and one for managing a form input on the Contact page.
- The Contact page must include a controlled input that captures and displays user input in real time.

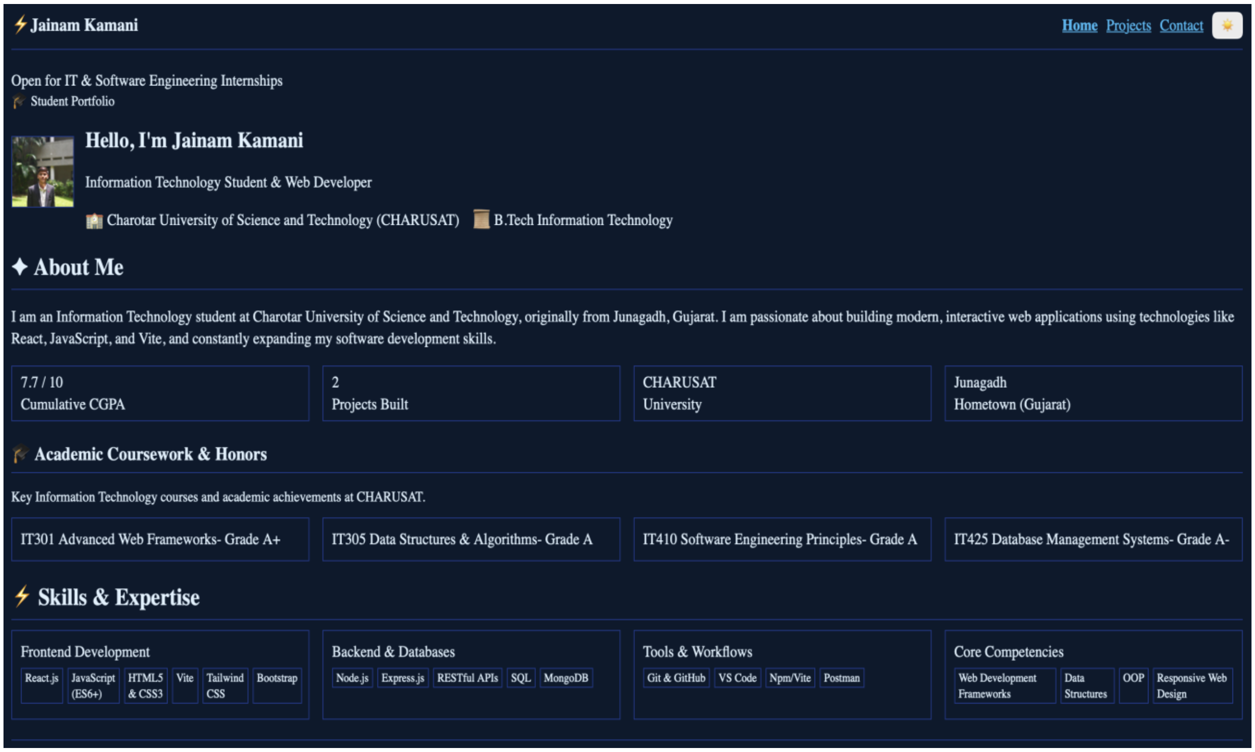
SUPPLEMENTARY PROBLEMS:

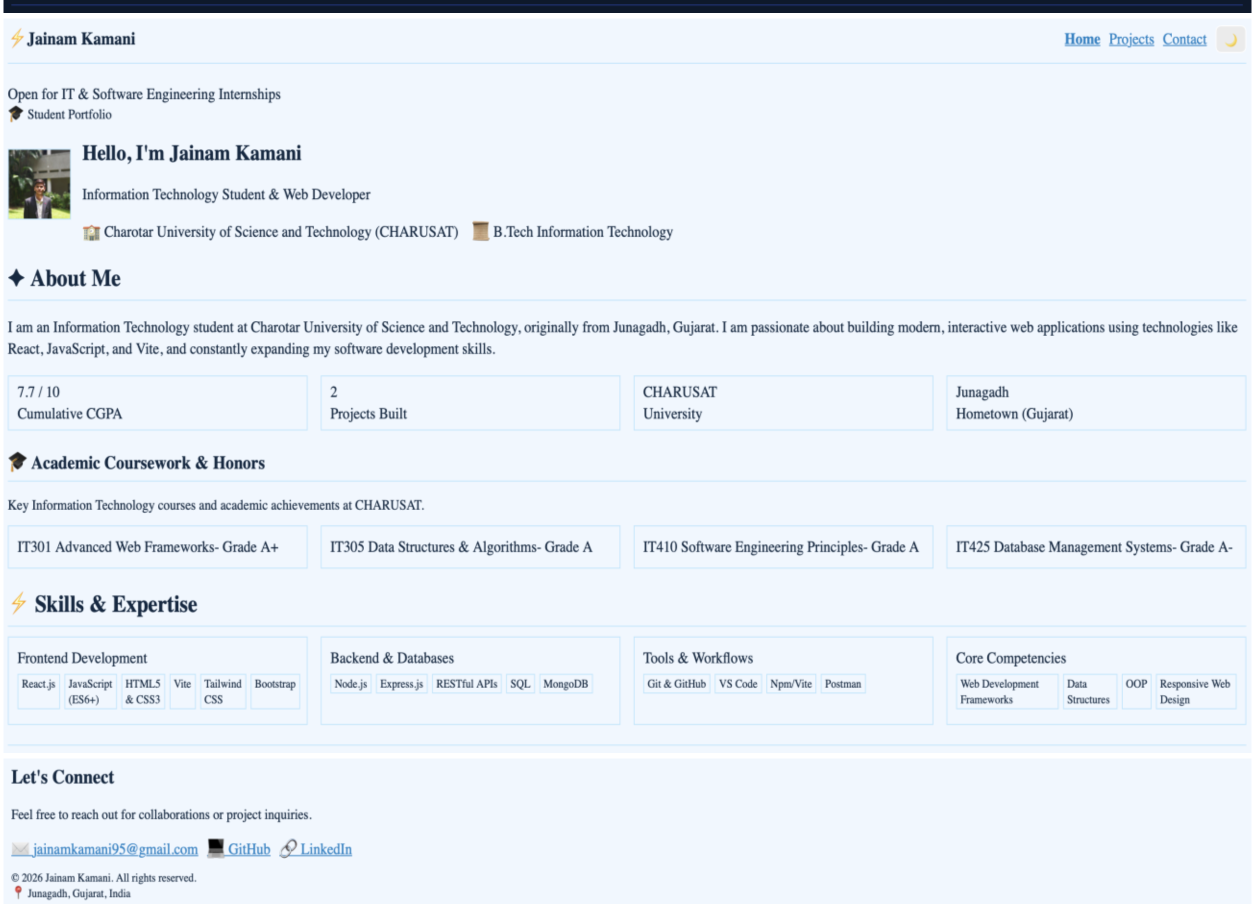
- Add a 404 Not Found route that renders a custom error component for unknown paths.
- Implement a dark/light mode toggle using useState that applies a CSS class to the root element.
- Store the contact form input in state and display a live character count below the input.

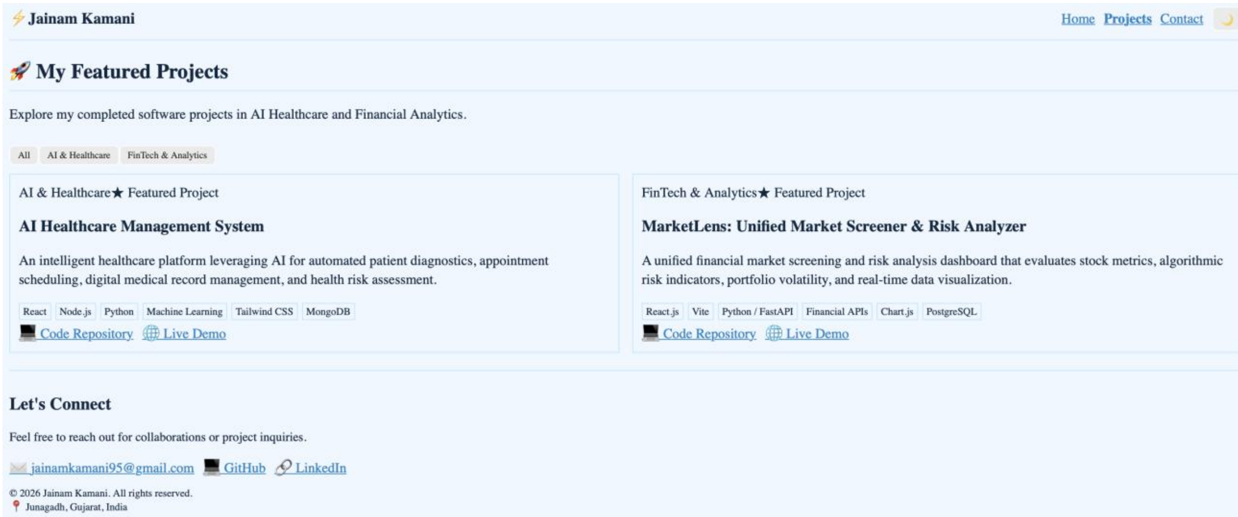
ARCHITECTURE / DIAGRAM:

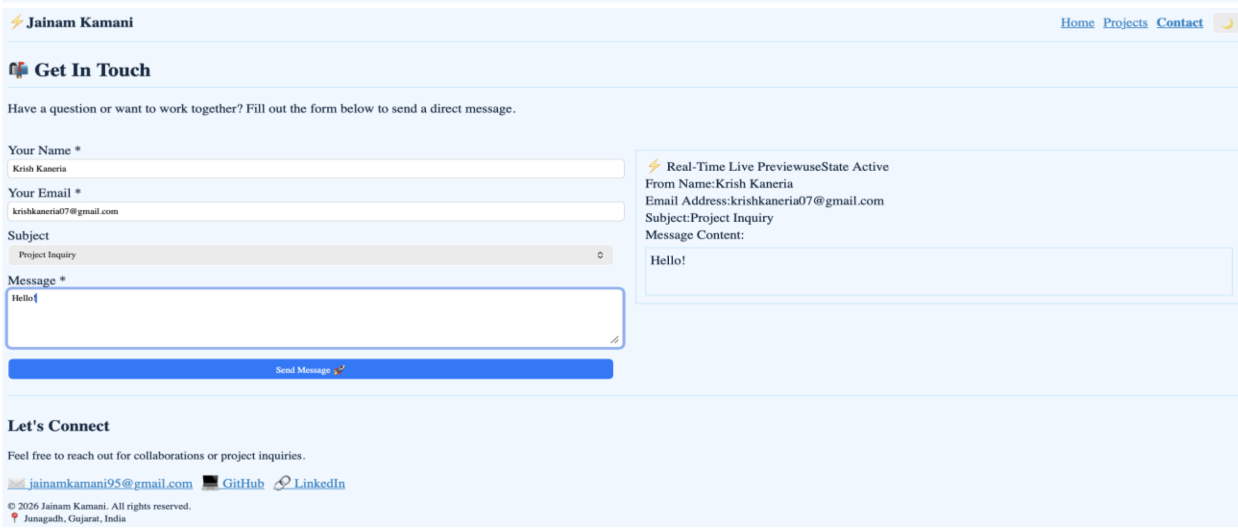


OUTPUT (SUPPLEMENTARY PROBLEMS INCLUDED):

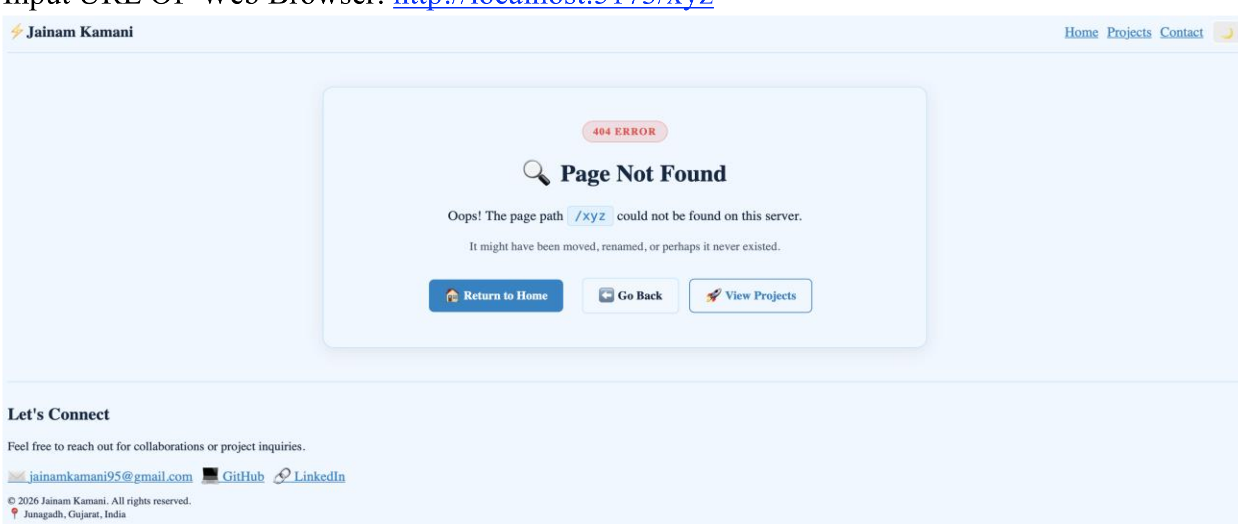








Original URL Of Web Browser: http://localhost:5173/
Input URL Of Web Browser: http://localhost:5173/xyz



KEY QUESTIONS / ANALYSIS:

1) What is the role of each component in the overall UI structure?

In React, a component is a reusable piece of the user interface. Each component has a specific responsibility, making the application organized and easier to manage.

For example, in an e-commerce website:

- Header Component – Displays the logo, navigation menu, and search bar.
- Sidebar Component – Shows categories or filters.
- Product Card Component – Displays a product's image, name, price, and button.
- Footer Component – Contains contact information, links, and copyright.

The main App component combines all these smaller components to build the complete webpage.

2) How does React re-render when props change?

Props (properties) are values passed from a parent component to a child component.

When a parent component passes new props to a child:

1. The parent component updates.
2. React compares the new props with the previous props.
3. If the props have changed, React re-renders the child component.
4. React updates only the parts of the webpage that actually changed using the Virtual DOM, making updates efficient.

Example:

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

function App() {
  return <Greeting name="Jainam" />;
}
```

If name changes from "Jainam" to "Rahul", React re-renders only the Greeting component with the updated name.

3) Why is component reusability important in large-scale applications?

Component reusability is important because it improves development speed and makes applications easier to maintain.

Benefits:

- Reduces duplicate code – Write a component once and use it multiple times.
- Easier maintenance – Updating one reusable component updates it everywhere it is used.
- Consistency – The same design and behavior are maintained throughout the application.
- Faster development – Developers can build new pages by combining existing components.
- Better teamwork – Different developers can work on different components independently.

Example:
A Button component can be reused for:

- Login button
- Sign Up button
- Buy Now button
- Submit button

Instead of writing button code repeatedly, you create one reusable component and customize it using props.

Example:

```
function Button({ text }) {
  return <button>{text}</button>; }

// Reuse
<Button text="Login" />
<Button text="Sign Up" />
<Button text="Buy Now" />
```

This makes React applications more scalable, easier to test, and simpler to maintain as they grow.

GITHUB REPOSITORY:

https://github.com/9081495207/ITUE301-Advanced-Web-Development-Frameworks